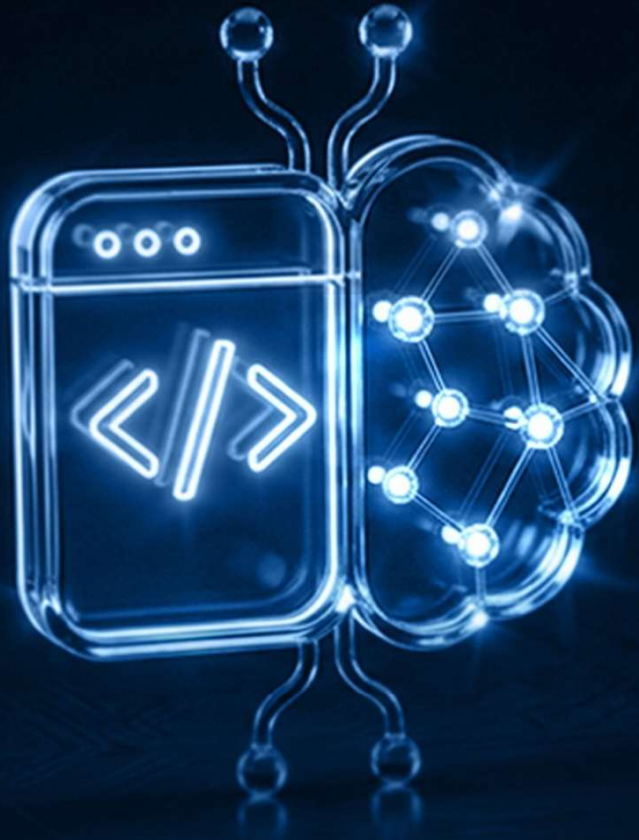


دوره آموزشی

GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

مدرس: **علیرضا ارومند**



جلسه پنجم

آشنایی با ابزارها و MCP

دوره آموزشی

GenAI

مهندسی نرم افزار در عصر هوش مصنوعی

مدرس: علیرضا ارومند





بخش اول —

از «مدل پاسخ‌گو» به «مدل
متصل به نرم‌افزار»



خروجی ساخت‌یافته، ابزار و MCP؛ اتصال مدل زبانی به نرم‌افزار

- مدل زبانی به‌تنهایی عمدتاً متن تولید می‌کند
- نرم‌افزار به داده، قرارداد و عملیات قابل کنترل نیاز دارد
- خروجی ساخت‌یافته، پل میان زبان و داده است
- ابزار، پل میان مدل و عملیات نرم‌افزاری است
- MCP، اتصال به قابلیت‌های بیرونی را استانداردتر می‌کند

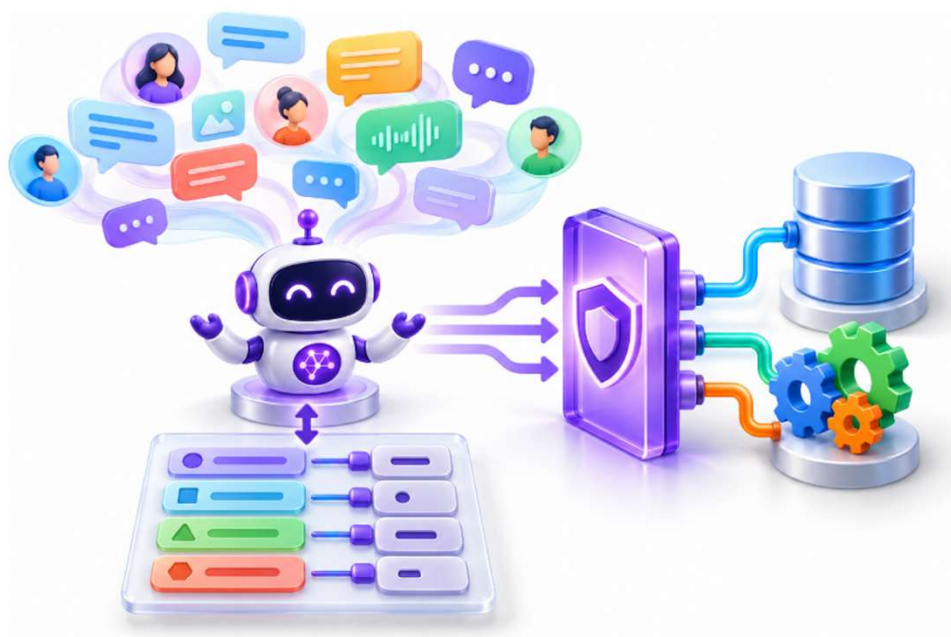


تا اینجا چه چیزی داشتیم؟

- در Spec Kit یاد گرفتیم رفتار مطلوب را دقیق تعریف کنیم
- در بحث مدل فهمیدیم خروجی مدل ذاتاً احتمالی است
- در Context فهمیدیم مدل فقط آنچه در دسترسش قرار می‌گیرد می‌بیند
- در Prompt Engineering رفتار مدل را هدایت کردیم، نه تضمین
- حالا باید مدل را وارد جریان واقعی نرم‌افزار کنیم



مسئله اصلی امروز چیست؟



- انسان با متن منعطف و توضیحی راحت است
- برنامه به ساختار قابل پیش‌بینی نیاز دارد
- مدل نمی‌تواند مستقیماً مجوز اجرای عملیات را تعیین کند
- نرم‌افزار نباید متن مدل را معادل فرمان اجرایی بداند
- اتصال مدل به سیستم، یک مسئله معماری است نه صرفاً Prompt



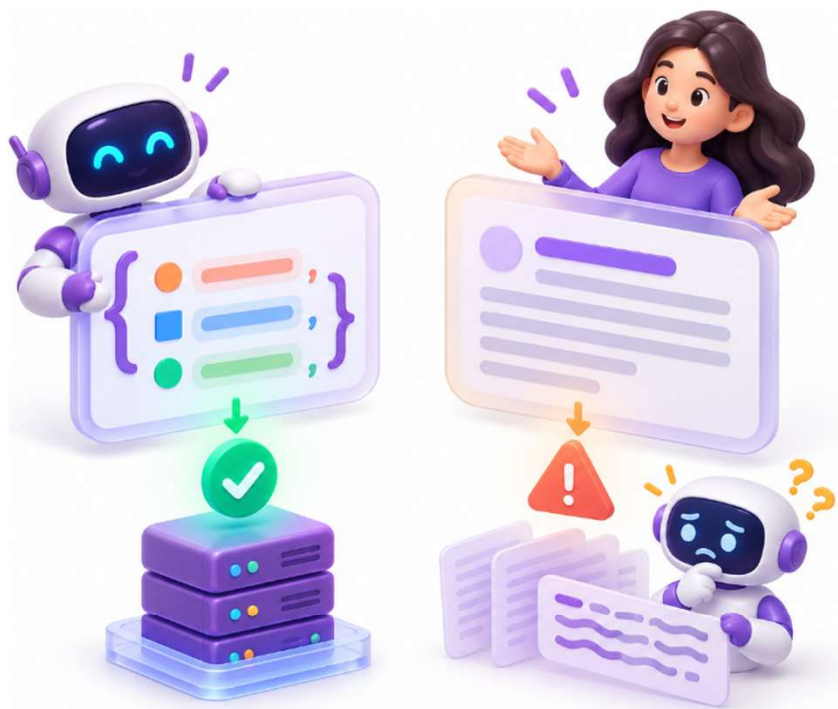


بخش دوم —

خروجی ساخت‌یافته؛ از متن
قابل خواندن به داده قابل
پردازش



متن خوب برای انسان، الزاماً خروجی خوب برای برنامه نیست



- «پست پیدا شد و فعلاً پیش‌نویس است» برای انسان کافی است
- برنامه به فیلدهایی مانند id و status نیاز دارد
- ترتیب و واژگان متن طبیعی ممکن است تغییر کنند
- Parsing متن آزاد شکننده و وابسته به عبارت‌بندی است
- مصرف ماشینی نیازمند قرارداد صریح است



خروجی ساخت‌یافته یعنی چه؟

- شکل خروجی پیش از تولید مشخص شده است
- هر بخش خروجی معنا و نوع مشخصی دارد
- برنامه می‌تواند بدون حدس آن را بخواند
- ساختار بخشی از قرارداد میان مدل و برنامه می‌شود
- JSON رایج است، اما اصل موضوع «قرارداد داده» است



JSON فقط ظرف است

- JSON ساختار را قابل نمایش می‌کند
- Object، Array، String، Number و Boolean نوع‌های متفاوت‌اند
- معتبر بودن JSON به معنای درست بودن پاسخ نیست
- JSON درباره قواعد کسب‌وکار چیزی نمی‌داند
- برای تعریف دقیق‌تر ساختار به Schema نیاز داریم



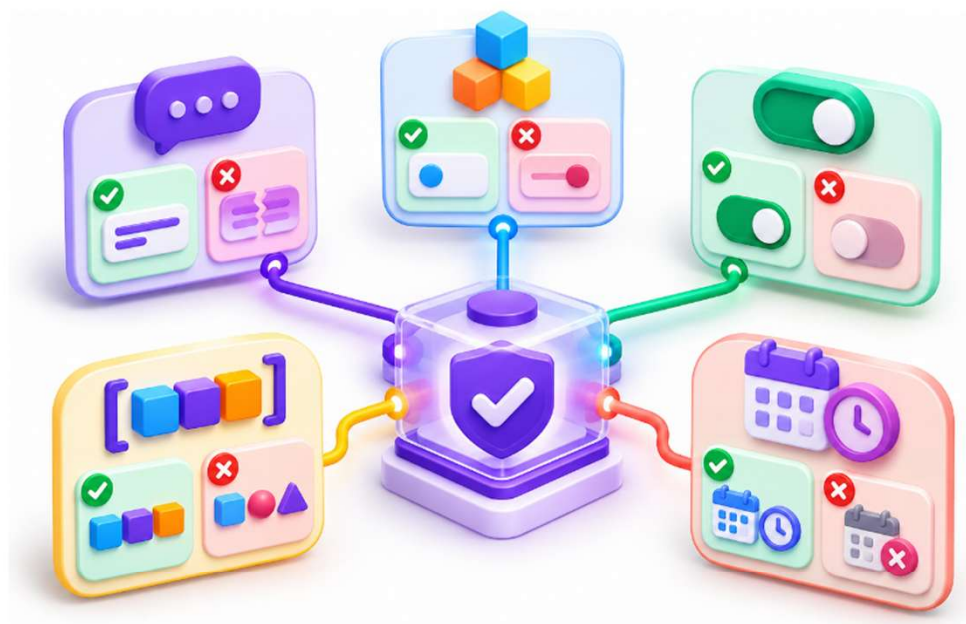
Schema؛ قرارداد شکل داده



- Schema مشخص می‌کند چه فیلدهایی وجود دارند
- نوع داده هر فیلد را تعریف می‌کند
- الزامی یا اختیاری بودن فیلدها را مشخص می‌کند
- می‌تواند دامنه مقادیر قابل قبول را محدود کند
- می‌تواند وجود فیلدهای اضافی را نیز کنترل کند



نوع داده فقط جزئیات فنی نیست



- شناسه عددی با متن آزاد یکی نیست
- تاریخ باید قرارداد مشخصی داشته باشد
- Boolean بهتر از پاسخهایی مثل «بله، تقریباً» است
- Array با یک مقدار منفرد رفتار متفاوتی دارد
- نوع نادرست باید قبل از ادامه جریان متوقف شود



مقادیر مجاز؛ وقتی هر String قابل قبول نیست

- بعضی فیلدها مجموعه محدودی از حالت‌ها دارند
- draft، published و archived نمونه وضعیت هستند
- متن آزاد احتمال تنوع و ابهام را بالا می‌برد
- Enum قرارداد میان مدل و برنامه را محدودتر می‌کند
- محدودسازی ساختار، تصمیم کسب‌وکاری را جایگزین نمی‌کند



«مفقود» با null یکی نیست

- نبودن فیلد یعنی داده اصلاً ارائه نشده است
- null یک مقدار صریح در داده است
- «نمی‌دانم» باید در قرارداد معنای مشخص داشته باشد
- مدل نباید برای پر کردن اجباری واقعیت بسازد
- طراحی Schema باید وضعیت اطلاعات ناقص را پیش‌بینی کند



وقتی پاسخ کامل نیست چه کنیم؟



- ناقص بودن باید قابل نمایش باشد، نه پنهان
- می‌توان وضعیت‌هایی مثل `input_needs` تعریف کرد
- فهرست داده‌های مفقود می‌تواند بخشی از پاسخ باشد
- برنامه تصمیم می‌گیرد ادامه ممکن است یا نه
- مدل نباید با حدس، قرارداد را ظاهراً کامل کند



از مدل بخواهیم JSON بدهد؛ کافی است؟



- Prompt می‌تواند قالب مورد انتظار را توضیح دهد
- مدل معمولاً تلاش می‌کند همان قالب را رعایت کند
- اما Prompt به‌تنهایی قراردادی اجرایی نیست
- ممکن است توضیح اضافی یا ساختار ناقص تولید شود
- «درخواست ساختار» با «اعمال ساختار» تفاوت دارد



سه سطح برای کنترل ساختار

- سطح اول: درخواست قالب فقط در Prompt
- سطح دوم: اعتبارسنجی خروجی و Retry یا Repair
- سطح سوم: سازوکار ساخت‌یافته بومی مدل یا محدودسازی تولید
- هر سطح احتمال خطای ساختاری را کاهش می‌دهد
- تضمین دقیق سطح سوم وابسته به سرویس و قرارداد همان سرویس است

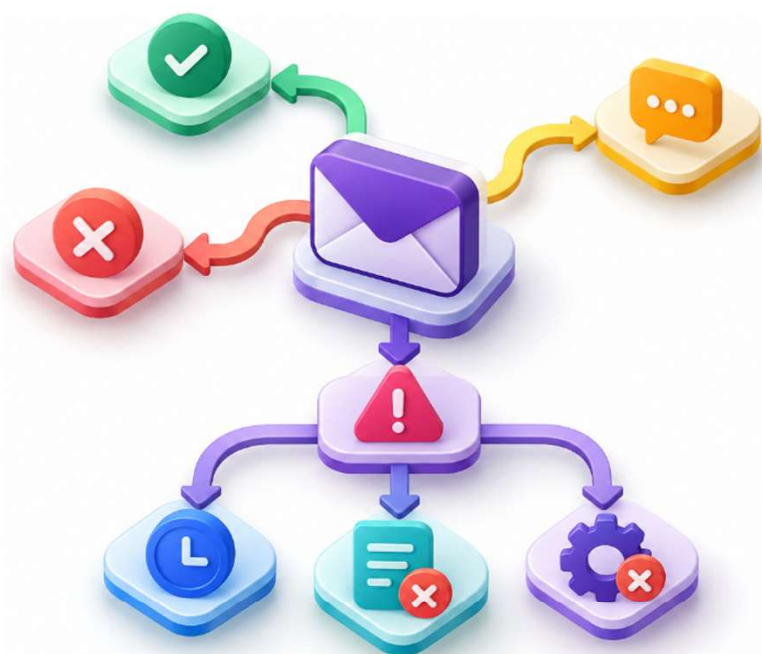


سه نوع «درست بودن» را قاطی نکنیم

- صحت نحوی: آیا خروجی قابل Parse است؟
- صحت ساختاری: آیا با Schema تطبیق دارد؟
- صحت معنایی: آیا مقادارها با واقعیت سازگارند؟
- اعتبار کسب و کاری: آیا عملیات در این وضعیت مجاز است؟
- عبور از یک سطح، سطح بعدی را تضمین نمی‌کند



امتناع، خطا و ناتوانی را داخل «موفقیت جعلی» نریزیم



- موفقیت، امتناع و خطای فنی وضعیت‌های متفاوت‌اند
- نبود داده با خرابی سرویس یکی نیست
- Timeout با «رکورد پیدا نشد» تفاوت دارد
- برنامه باید این حالت‌ها را جداگانه مدیریت کند
- قرارداد خروجی باید مسیر شکست را هم تعریف کند





بخش سوم —

Function و Tool Use
Calling؛ مدل چگونه
درخواست عملیات می‌کند؟



GenAI

مدل اطلاعات سیستم ما را از کجا می‌آورد؟



- دانش مدل جای دادهٔ زندهٔ سیستم را نمی‌گیرد
- مدل از وضعیت فعلی دیتابیس ما خبر ندارد
- Context دادن همهٔ اطلاعات همیشه عملی نیست
- گاهی مدل باید اطلاعات موردنیاز را درخواست کند
- اینجاست که مفهوم Tool وارد می‌شود



Tool چیست؟

- Tool یک قابلیت مشخص و قابل فراخوانی است
- نام، توضیح و قرارداد آرگومان دارد
- می‌تواند داده بخواند یا عملیاتی انجام دهد
- مدل درباره استفاده از آن استدلال می‌کند
- اجرای واقعی Tool خارج از خود مدل انجام می‌شود



Function Calling دقیقاً چه چیزی را عوض می‌کند؟

- مدل به جای نوشتن دستور در متن، درخواست ساخت‌یافته می‌دهد
- درخواست شامل نام Function یا Tool است
- آرگومان‌ها نیز ساخت‌یافته تولید می‌شوند
- برنامه درخواست را دریافت و بررسی می‌کند
- مدل Function داخل برنامه ما را مستقیماً اجرا نمی‌کند



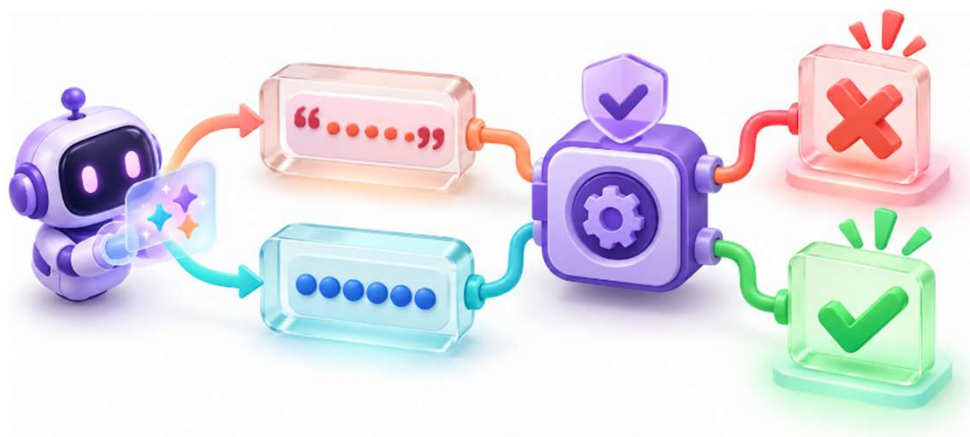
انتخاب Tool خودش یک تصمیم مدل است



- چند Tool ممکن است همزمان در دسترس باشند
- توضیح Tool روی انتخاب مدل اثر می‌گذارد
- نام‌های مبهم احتمال انتخاب اشتباه را افزایش می‌دهند
- Tool‌های هم‌پوشان تصمیم‌گیری را سخت‌تر می‌کنند
- طراحی Catalog ابزار بخشی از معماری است



آرگومان Tool هم باید قرارداد داشته باشد



- مدل نام Tool را انتخاب می‌کند
- سپس آرگومان مناسب آن را تولید می‌کند
- نوع و الزام آرگومان‌ها باید مشخص باشد
- آرگومان ساختاریافته هنوز قابل اعتماد مطلق نیست
- برنامه باید قبل از اجرا آن را اعتبارسنجی کند



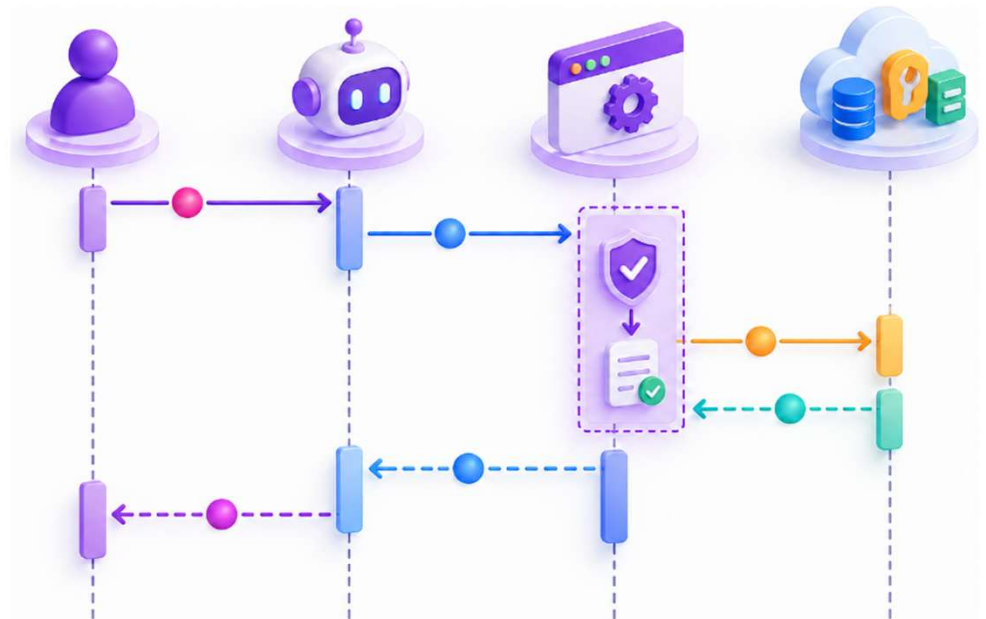
Tool Call پاسخ نهایی کاربر نیست

- Tool Call یک درخواست میانی در جریان است
- کاربر معمولاً نتیجه عملیات را می‌خواهد
- برنامه Tool را اجرا می‌کند
- نتیجه دوباره در اختیار مدل قرار می‌گیرد
- مدل با استفاده از نتیجه پاسخ را ادامه می‌دهد



چرخه کامل Tool Use

- درخواست کاربر وارد مدل می‌شود
- مدل Tool و آرگومان را پیشنهاد می‌دهد
- برنامه Policy و آرگومان را بررسی می‌کند
- Tool در سیستم واقعی اجرا می‌شود
- نتیجه به مدل برمی‌گردد و پاسخ ادامه پیدا می‌کند



برنامه می‌تواند Tool Call مدل را رد کند



- مدل مرجع نهایی مجوز نیست
- Tool ممکن است برای کاربر مجاز نباشد
- آرگومان می‌تواند خارج از محدوده باشد
- وضعیت فعلی سیستم ممکن است عملیات را منع کند
- درخواست مدل فقط یکی از ورودی‌های تصمیم برنامه است



Tool خواندنی و Tool تغییردهنده یک ریسک ندارند



- `post_get` صرفاً اطلاعات می‌خواند
- `post_publish` وضعیت سیستم را تغییر می‌دهد
- عملیات تغییردهنده ممکن است برگشت‌ناپذیر باشد
- سطح تأیید و مجوز باید متناسب با اثر باشد
- طراحی Tool باید اثر جانبی را شفاف کند



Timeout و Retry؛ یک خطای ساده می‌تواند خطرناک شود

- Tool ممکن است به‌موقع پاسخ ندهد
- Timeout لزوماً به معنای اجرا نشدن عملیات نیست
- Retry روی عملیات خواندنی معمولاً کم‌خطرتر است
- تکرار عملیات تغییردهنده می‌تواند اثر دوباره بسازد
- سیاست Retry باید از ماهیت عملیات آگاه باشد

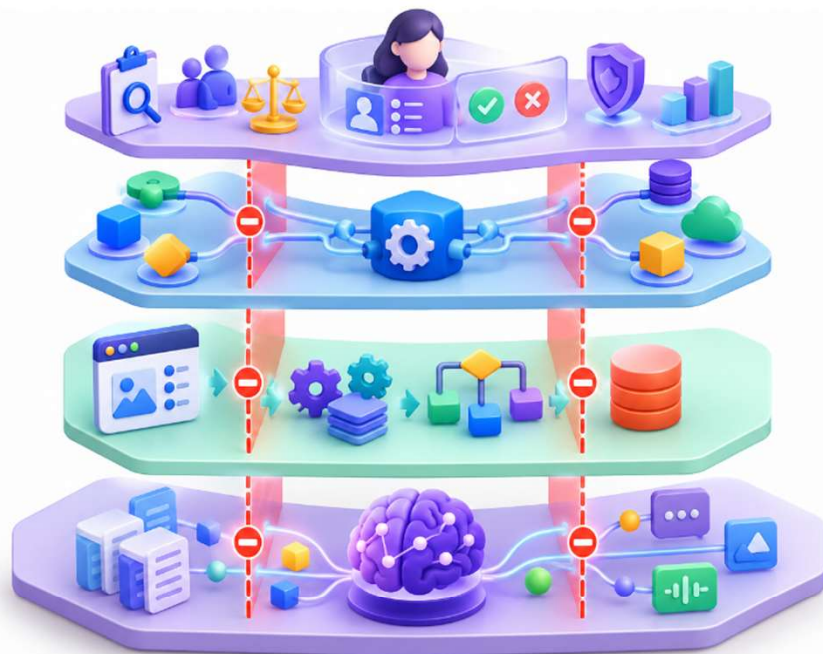


Idempotency؛ جلوگیری از اثر تکراری

- یک درخواست منطقی نباید چند اثر کسب و کاری بسازد
- عملیات حساس می‌تواند Idempotency Key داشته باشد
- برنامه باید درخواست تکراری را تشخیص دهد
- Retry و Idempotency باید با هم طراحی شوند
- مدل نباید مسئول جلوگیری از تکرار عملیات باشد



مرز مسئولیت‌ها را از همین‌جا روشن کنیم



- مدل: فهم درخواست و پیشنهاد Tool و آرگومان
- برنامه: Policy، Validation، Timeout و Orchestration
- API: اجرای واقعی و اعمال قواعد دامنه
- انسان: تصمیم یا تأیید در نقاط پرسیک
- انتقال مسئولیت کنترل به مدل یک خطای معماری است

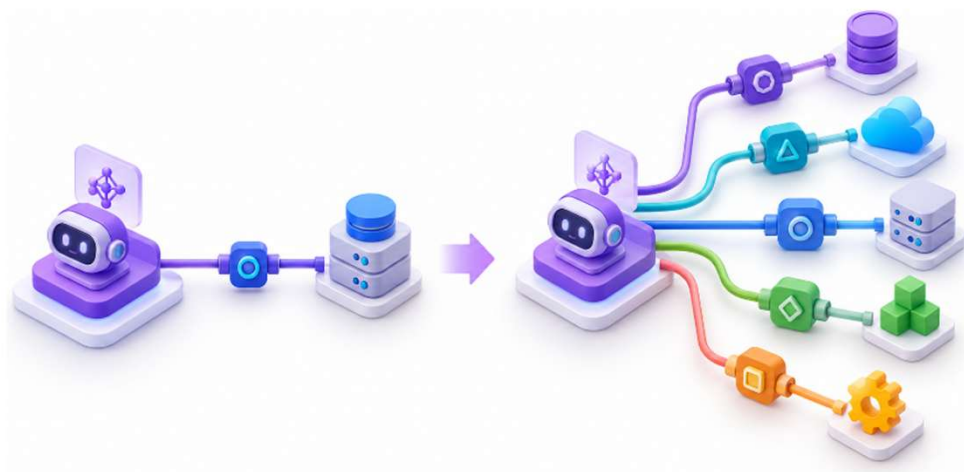




بخش چهارم —
MCP؛ استاندارد کردن اتصال
قابلیت‌ها به برنامه‌های
هوشمند



اگر فقط یک Tool داشته باشیم، مسئله ساده است

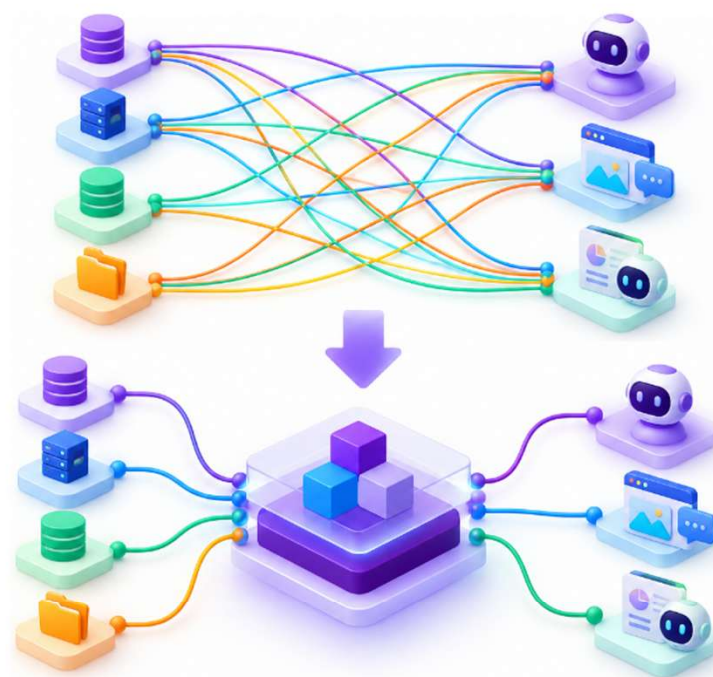


- یک برنامه و یک API را مستقیم متصل می‌کنیم
- Tool Definition را خودمان می‌نویسیم
- Mapping و Authentication را مدیریت می‌کنیم
- نتیجه را به مدل باز می‌گردانیم
- اما با زیاد شدن سیستم‌ها هزینه Integration رشد می‌کند



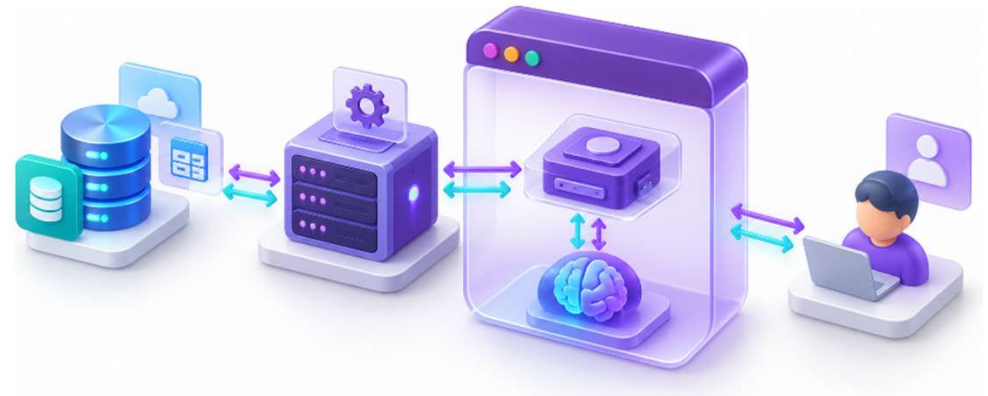
مسئله‌ای که MCP می‌خواهد حل کند

- برنامه‌های هوشمند به منابع و ابزارهای متعدد نیاز دارند
- هر Integration اختصاصی قرارداد متفاوتی ایجاد می‌کند
- کشف قابلیت‌ها معمولاً باید جداگانه طراحی شود
- MCP یک پروتکل مشترک برای این اتصال تعریف می‌کند
- MCP جای منطق کسب‌وکار یا API‌های موجود را نمی‌گیرد



معماری MCP؛ Host، Client و Server

- Host همان برنامه هوشمندی است که کاربر با آن کار می‌کند
- MCP Client داخل Host ارتباط MCP را مدیریت می‌کند
- MCP Server قابلیت‌ها را در اختیار Client قرار می‌دهد
- Server می‌تواند پشت خود API، Database یا سرویس دیگری باشد
- مدل همچنان مرکز اجرای مستقیم عملیات نیست



MCP Server چه چیزی ارائه می‌دهد؟



- Tools برای انجام عملیات یا محاسبات
- Resources برای ارائه داده و Context
- Prompts برای الگوهای پیام و Workflow
- Server می‌تواند فقط بخشی از این قابلیت‌ها را ارائه کند
- این Primitive ها نقش یکسانی ندارند



MCP در Tool



- Tool یک قابلیت قابل فراخوانی توسط مدل است
- Tool نام و Schema ورودی دارد
- Server فهرست Tool های قابل دسترس را اعلام می کند
- Client Tool مناسب را در اختیار جریان مدل قرار می دهد
- اجرای واقعی پشت MCP Server انجام می شود



Resource در MCP با Tool یکی نیست

- Resource عمدتاً برای ارائه اطلاعات و Context است
- Tool نماینده یک قابلیت قابل فراخوانی است
- خواندن سند و اجرای عملیات دو مفهوم متفاوت‌اند
- انتخاب Primitive مناسب قرارداد را روشن‌تر می‌کند
- همه داده‌ها را نباید به Tool تبدیل کرد

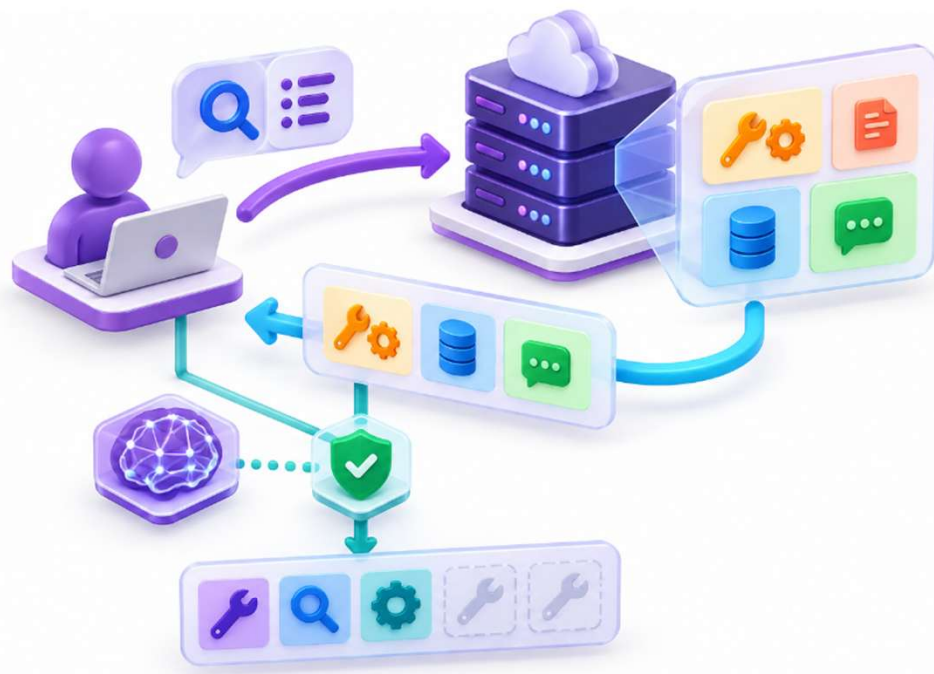


Prompt در MCP چه نقشی دارد؟

- Prompt یک الگوی پیام یا Workflow قابل ارائه است
- Prompt خودش اجرای عملیات نیست
- Server می‌تواند الگوهای تخصصی دامنه را عرضه کند
- ورودی Prompt باید مانند ورودی‌های دیگر اعتبارسنجی شود
- Prompt جای Authorization و Business Rule را نمی‌گیرد



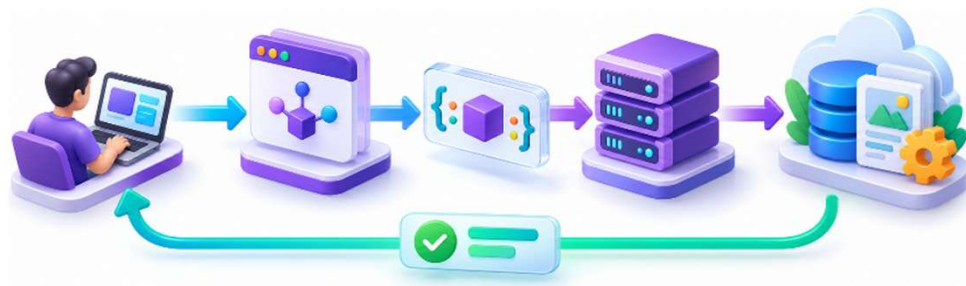
کشف قابلیت‌ها؛ لازم نیست همه‌چیز از قبل Hard-code شود



- Client باید بداند Server چه قابلیت‌هایی ارائه می‌کند
- Toolها و سایر Primitiveها قابل فهرست شدن هستند
- قابلیت‌های در دسترس می‌توانند با مجوز متفاوت باشند
- کشف قابلیت، اتصال را پویاتر می‌کند
- کشف Capability به معنای مجاز بودن اجرای همهٔ آنها نیست



مسیر ارتباط در MCP؛ فقط به اندازه‌ای که لازم داریم

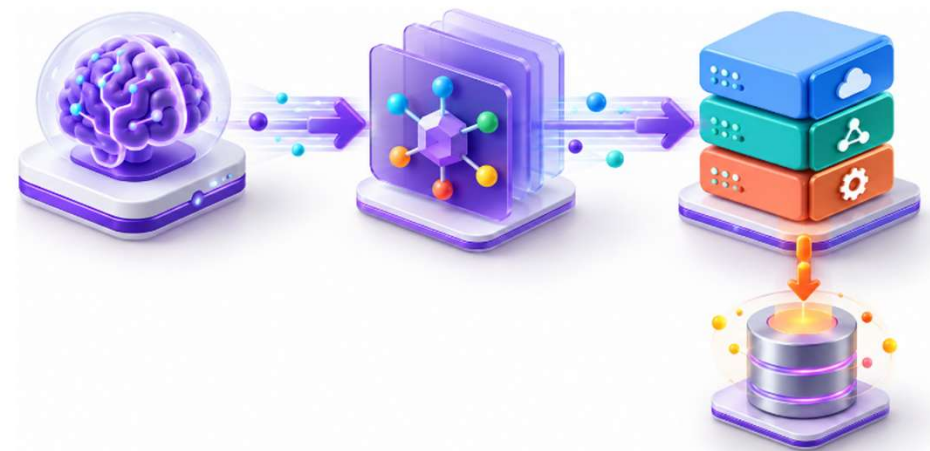


- MCP قرارداد پیام میان Client و Server را استاندارد می‌کند
- پروتکل بر پیام‌های JSON-RPC تکیه دارد
- درخواست Tool از Host به Server منتقل می‌شود
- Server نتیجه ساخت‌یافته را بازمی‌گرداند
- جزئیات Transport با محل منطق کسب‌وکار یکی نیست



API، Function Calling و MCP سه چیز متفاوت‌اند

- API قرارداد سرویس نرم‌افزاری واقعی است
- Function Calling سازوکار مدل برای درخواست Tool است
- MCP قرارداد استاندارد میان Host و ارائه‌دهنده قابلیت است
- MCP Server ممکن است در پشت خود یک API را فراخوانی کند
- استاندارد شدن اتصال، اجرای واقعی عملیات را جابه‌جا نمی‌کند





بخش پنجم —
اعتماد، امنیت و کنترل
اجرای

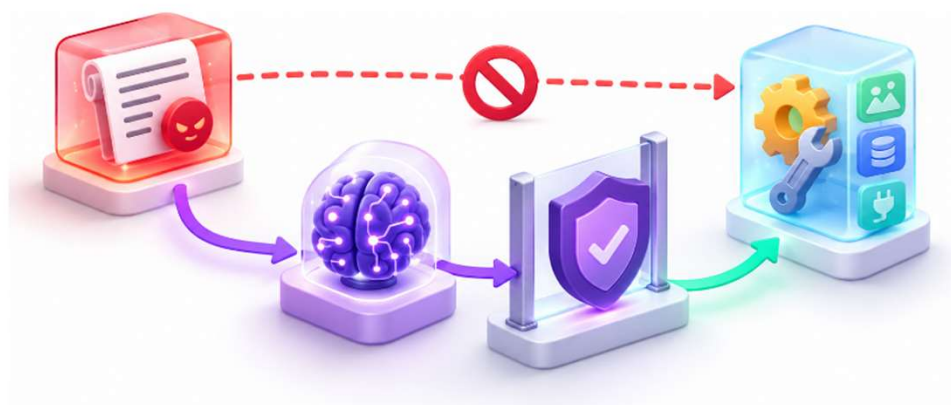


نتیجه Tool هم «حقیقت مطلق» نیست

- Tool می‌تواند خطا کند یا داده قدیمی برگرداند
- سرویس پایین‌دستی ممکن است Partial Result بدهد
- نوع و منشأ نتیجه باید بررسی شود
- نتیجه Tool باید متناسب با حساسیت عملیات اعتبارسنجی شود
- اعتماد به Tool بخشی از مدل اعتماد سیستم است



Prompt Injection می‌تواند از داخل داده وارد شود



- محتوای بیرونی ممکن است شامل دستور متنی باشد
- متن یک سند نباید خودکار تبدیل به Instruction شود
- Tool Result می‌تواند حامل محتوای مخرب باشد
- داده و دستور باید در معماری از هم تفکیک شوند
- مدل به‌تنهایی مرز امنیتی قابل اتکا نیست



کنترل باید در برنامه اعمال شود، نه در Prompt



- Prompt می‌تواند رفتار مطلوب را توضیح دهد
- Permission باید در لایه اجرایی بررسی شود
- Validation باید قبل از Tool Execution انجام شود
- عملیات حساس می‌تواند تأیید انسانی بخواند
- Audit و Logging باید مستقل از همکاری مدل باشند



معماری نهایی؛ مدل تصمیم‌یار است، نه مرجع نهایی کنترل

- Structured Output ارتباط داده‌ای را قابل اتکاتر می‌کند
- Tool Use مدل را به قابلیت‌های واقعی متصل می‌کند
- MCP اتصال قابلیت‌ها را استاندارد می‌کند
- برنامه همچنان مرجع Policy و Orchestration باقی می‌ماند
- API و Domain مرجع نهایی قواعد و تغییر وضعیت هستند
- انسان در نقاط پیرریسک حلقه کنترل را کامل می‌کند

